OTUS

Kotlin Backend Developer.Professional

otus.ru

REC

Проверить, идет ли запись

Меня хорошо видно && слышно?

RabbitMQ

Правила вебинара

Активно
участвуем

Off-topic обсуждаем в
OTUS-Kotlin-2025-05

Задаем вопрос
в чат или голосом

Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения

Индивидуально

Время, необходимое
на активность

Пишем в чат

Говорим голосом

Документ

Ответьте себе или
задайте вопрос

О курсе

Модуль 1: Введение в Kotlin	Введение	Базовые элементы	Функц. элементы	ООП		
Модуль 2: Расширенные возможности Kotlin	DSL	Корутины	Sequence, Flow	Multi-platform	Interop	Gradle
Модуль 3: Подготовка к разработке	Выбор проекта	Маркетинг аналитика	Security	Архитек-тура	DevOps	
Модуль 4: Разработка API	OpenAPI	Модели и маперы	Testing	Monitoring		
Модуль 5: Фреймворки и асинхронный транспорт	Spring	Ktor	WebSocket	RabbitMQ	Kafka	Serverless
Модуль 6: Бизнес-логика проекта	Основы БЛ	Модуль БЛ	Валидация			
Модуль 7: Работа с базами данных	Работа с БД	Репози-торий	Конкурен-ция	Postgres	Cassandra	Gremlin
Модуль 8: Развитие бизнес-логики проекта	Машина состояний	Auth	Основы ML	ML в бэкенде	Обзор экосист.	
Модуль 9: Проектная работа	Защита					

Окатов Сергей
автор программы, руководитель курса

- кандидат физ.-мат. наук
- 15 лет научной работы в университетах РФ, Европы, США
- 10 лет комм. разработки в стартапах и корпорациях (Сбер, Ланит) в качестве руководителя разработки, архитектора
- Специализация
  - Котлин
  - Большие данные
  - Распределенные системы
  - Архитектура

План занятия

- Общая информация о RabbitMQ
- Сценарии работы с RabbitMQ
- Подключение в проект
- Интеграция с логикой
- Рефлексия

Цели вебинара

После занятия вы:

1. Будете знать особенности RabbitMQ и сможете принимать решение о его применении в проекте
2. Узнаете, как взаимодействовать с RabbitMQ из кода
3. Сможете интегрировать транспортную часть, взаимодействующую с RabbitMQ и бизнес-логику в приложении

Смысл

Зачем вам это уметь

1. RabbitMQ широко используется в качестве транспорта в различных проектах, использующих сервисную архитектуру
2. RabbitMQ имеет определенные преимущества и недостатки, о которых нужно знать, чтобы взвешенно принимать решение о его применимости в конкретном случае

Системы обмена сообщениями (MQ): теория

Message broker / Брокер сообщений

Брокер сообщений

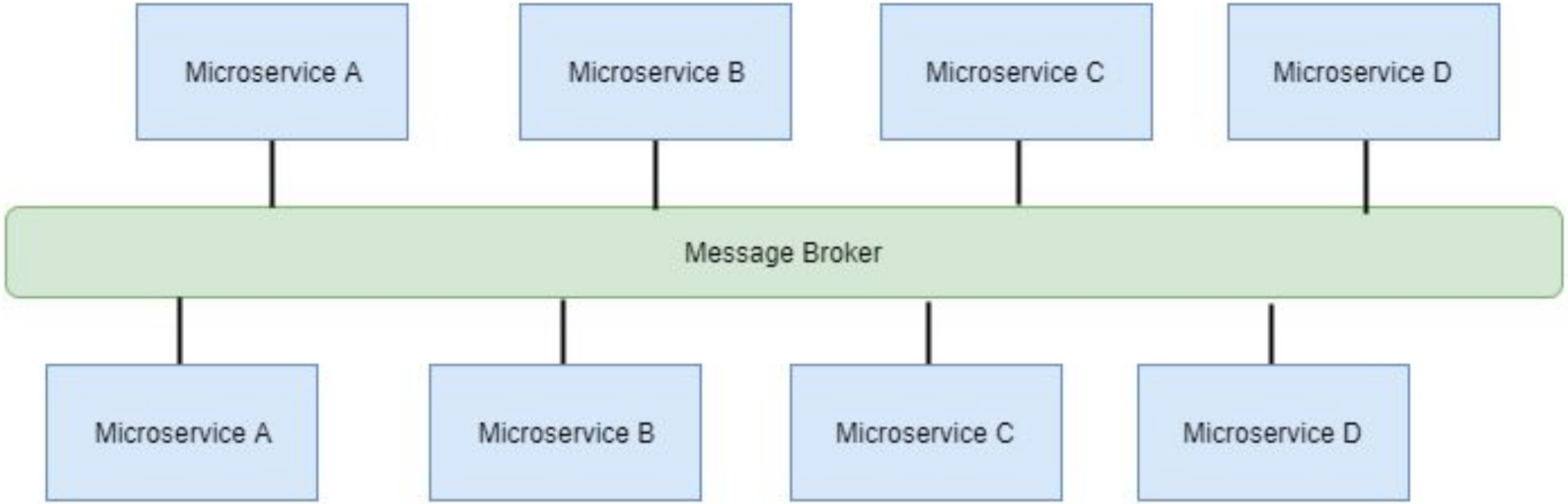
Материал из Википедии — свободной энциклопедии

Брокер сообщений (англ. *message broker*, *integration broker*, *interface engine*) — архитектурный паттерн в распределённых системах; приложение, которое преобразует сообщение по одному протоколу от приложения-источника в сообщение протокола приложения-приёмника, тем самым выступая между ними посредником. Кроме преобразования сообщений из одного формата в другой, в задачи брокера сообщений также входит:

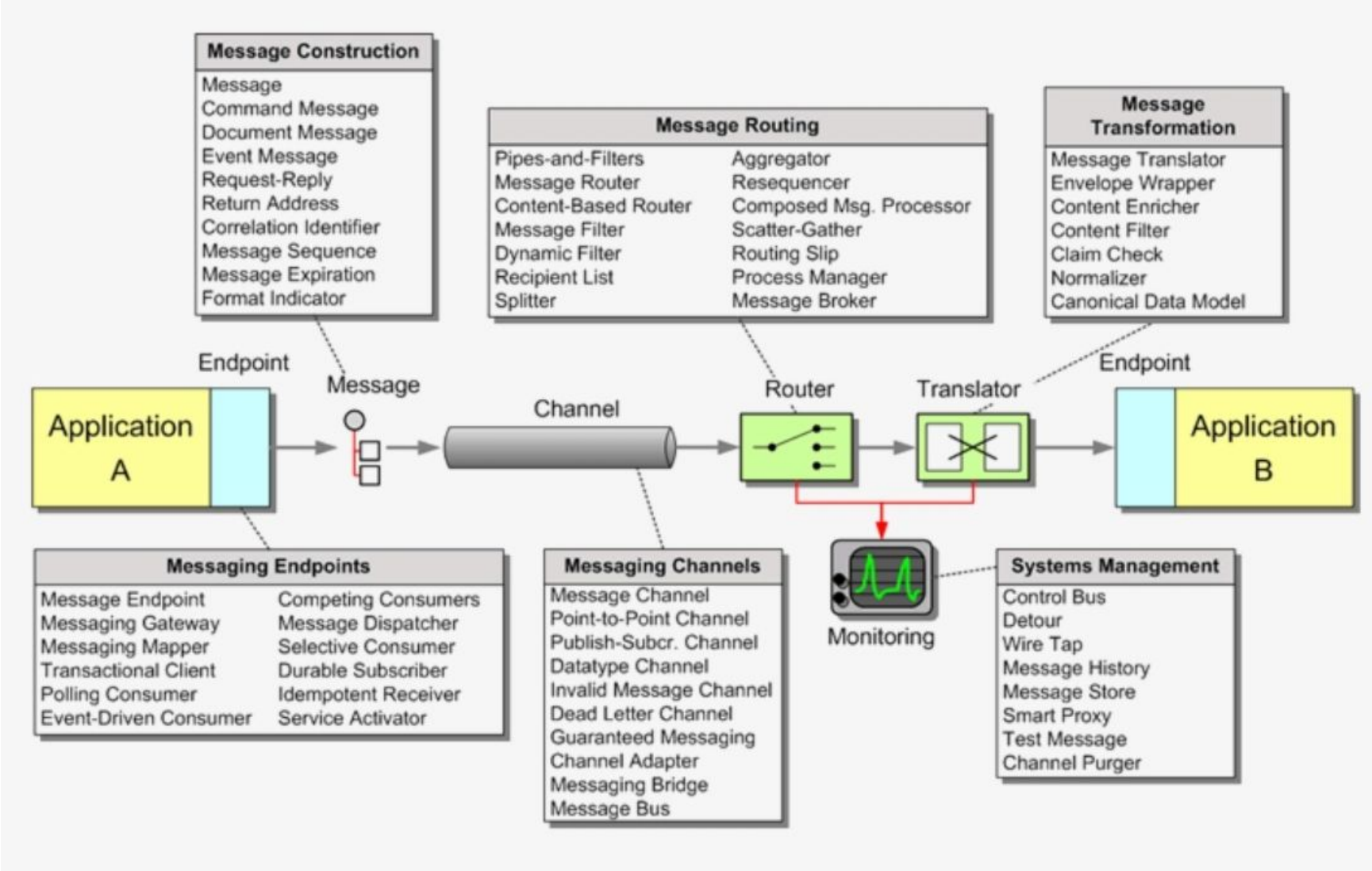
- проверка сообщения на ошибки;
- маршрутизация конкретному приемнику(ам);
- разбиение сообщения на несколько маленьких, а затем агрегирование ответов приёмников и отправка результата источнику;
- сохранение сообщений в базе данных;
- вызов веб-сервисов;
- распространение сообщений подписчикам, если используются шаблоны типа «издатель — подписчик».

Использование брокеров сообщений позволяет разгрузить веб-сервисы в распределённой системе, так как при отправке сообщений им не нужно тратить время на некоторые ресурсоёмкие операции типа маршрутизации и поиска приёмников. Кроме того, брокер сообщений для повышения эффективности может реализовывать стратегии упорядоченной рассылки и определение приоритетности, балансировать нагрузку и прочее.

MQ as Enterprise Service Bus



Паттерны обмена сообщениями

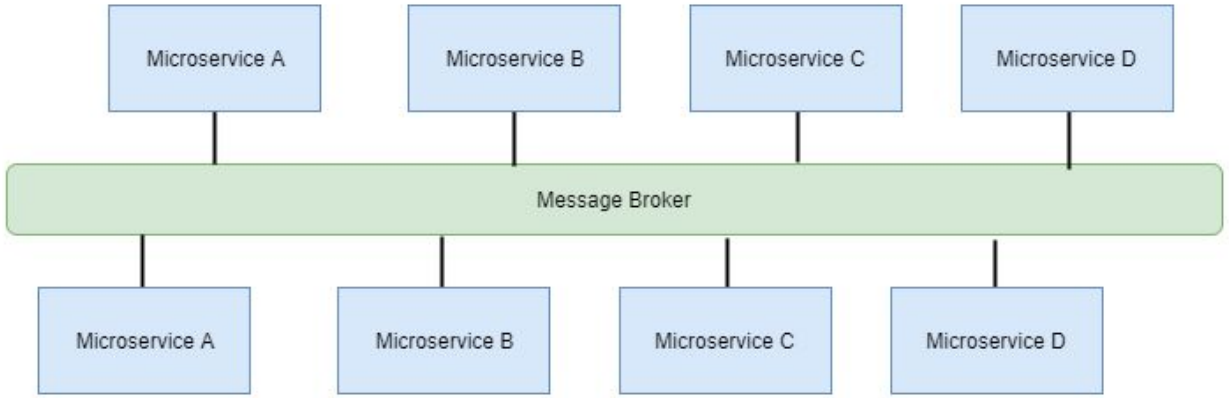


https://www.enterpriseintegrationpatterns.com/patterns/messaging/

Плюсы MQ

Особенности такого решения:

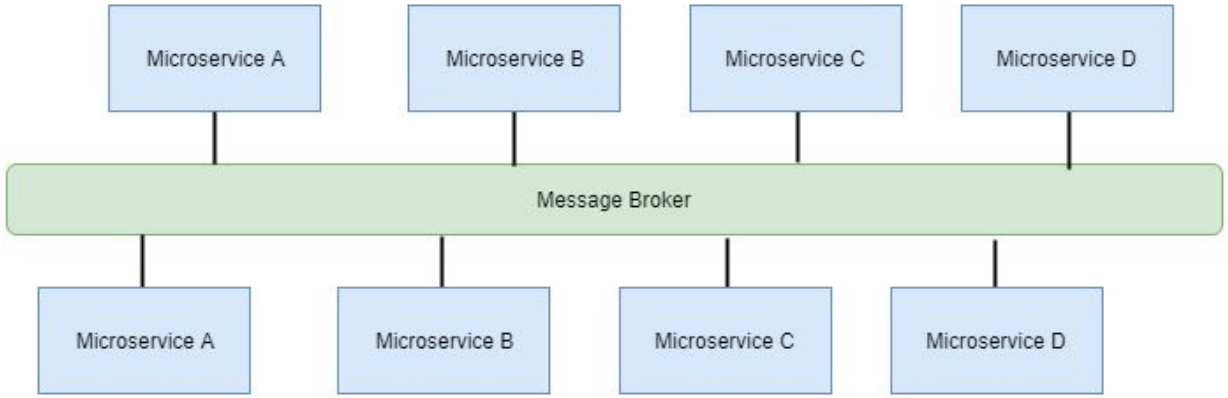
- Service discovery - сервисам не нужно знать где и кто находится, достаточно просто бросить сообщение в очередь
- Выше доступность – если сервис упал, то он просто не берет запросы из очереди, берет тот, который работает
- Ниже вероятность каскадных падений (сообщение полежит в очереди, пока получатель не поднимется)
- Горизонтальное масштабирование до некоторых пределов



Минусы MQ

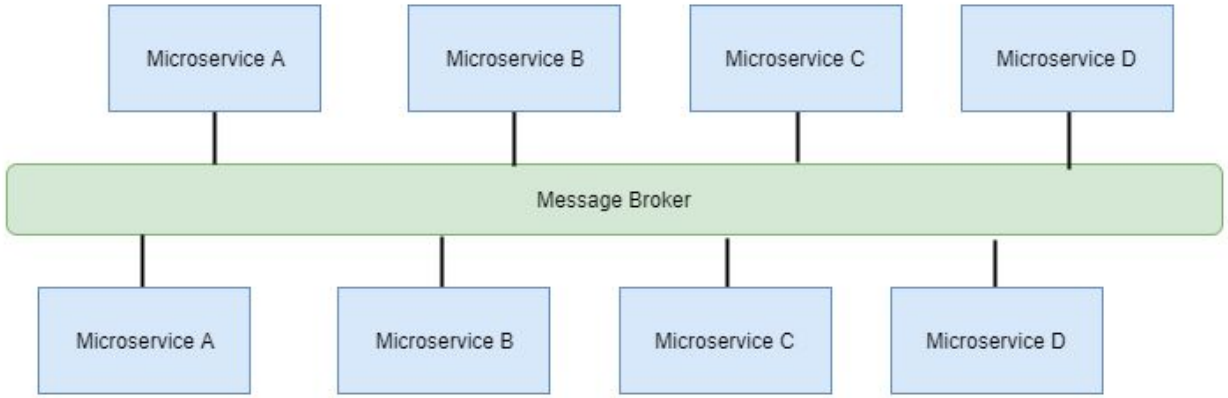
Минусы message bus

- Latency увеличивается по сравнению с синхронными вызовами (за счет накладных расходов на дополнительные сетевые взаимодействия с участием брокера). *Несущественно на фоне тормозов БД и при учете других систем service discovery.*
- Debug становится сложнее. *Все проблемы легко решаются.*
- Single Point Of Failure в виде брокера сообщений. *Сейчас это не сингл-поинт, а распределенная система*

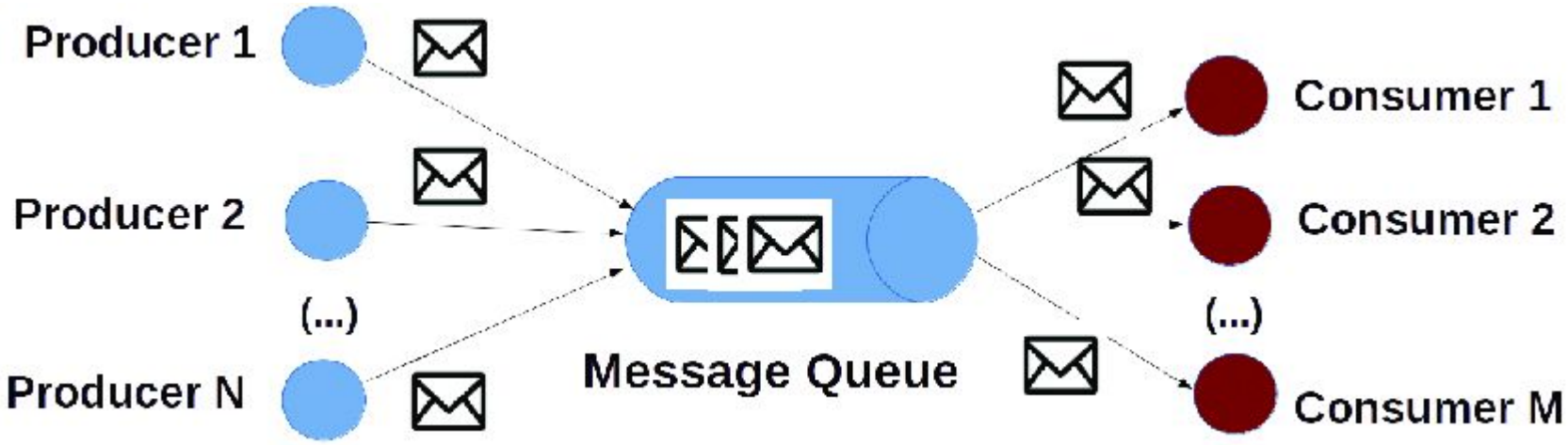


Особенности MQ в современности

- Реализует Event-Driven модель.
- В современном мире происходит планомерный отказ от синхронных взаимодействий в пользу асинхронных.
- Любой синхронный обмен — это дополнительный расход памяти
- В некоторых случаях без синхронного обмена не обойтись. Пример: работа с БД для хранения состояния в stateless-системах.
- MQ обеспечивает как асинхронное взаимодействие, так и синхронное (RPC).
- MQ не предназначены для обмена большими сообщениями.
- HTTP/2, HTTP/3 уже реализуют асинхронную модель (границы HTTP-MQ стираются)



Взаимодействие через MQ



Очень похоже на взаимодействие по HTTP через балансировщик

В теле бинарные данные

- JSON
- XML
- Protobuf
- Thrift
- Avro

```
message Person {
  required string name = 1;
  required int32 id = 2;
  optional string email = 3;
}
```

```
struct ListBonks {
  1: list<Bonk> bonk
}
struct NestedListsBonk {
  1: list<list<list<Bonk>>> bonk
}

struct BoolTest {
  1: optional bool b = true;
  2: optional string s = "true";
}
```

```
{"namespace": "example.avro",
 "type": "record",
 "name": "User",
 "fields": [
     {"name": "name", "type": "string"},
     {"name": "favorite_number",  "type": ["int", "null"]},
     {"name": "favorite_color", "type": ["string", "null"]}
 ]
}
```

О RabbitMQ

Общая информация о RabbitMQ

- Брокер сообщений
- Назначение – доставка сообщений от продюсера к консьмеру
- Поддерживает протоколы AMQP 0-9-1, AMQP 1.0, MQTT 3.1 via a plugin и STOMP (WS)
- Написан на Erlang
- AMQP предложен в 2003 John O'Hara из JPMorgan Chase в Лондоне
- Куплен в 2010 SpringSource (подразделение VMware)
- В 2013 стал частью Pivotal Software

Брокеры, поддерживающие AMQP

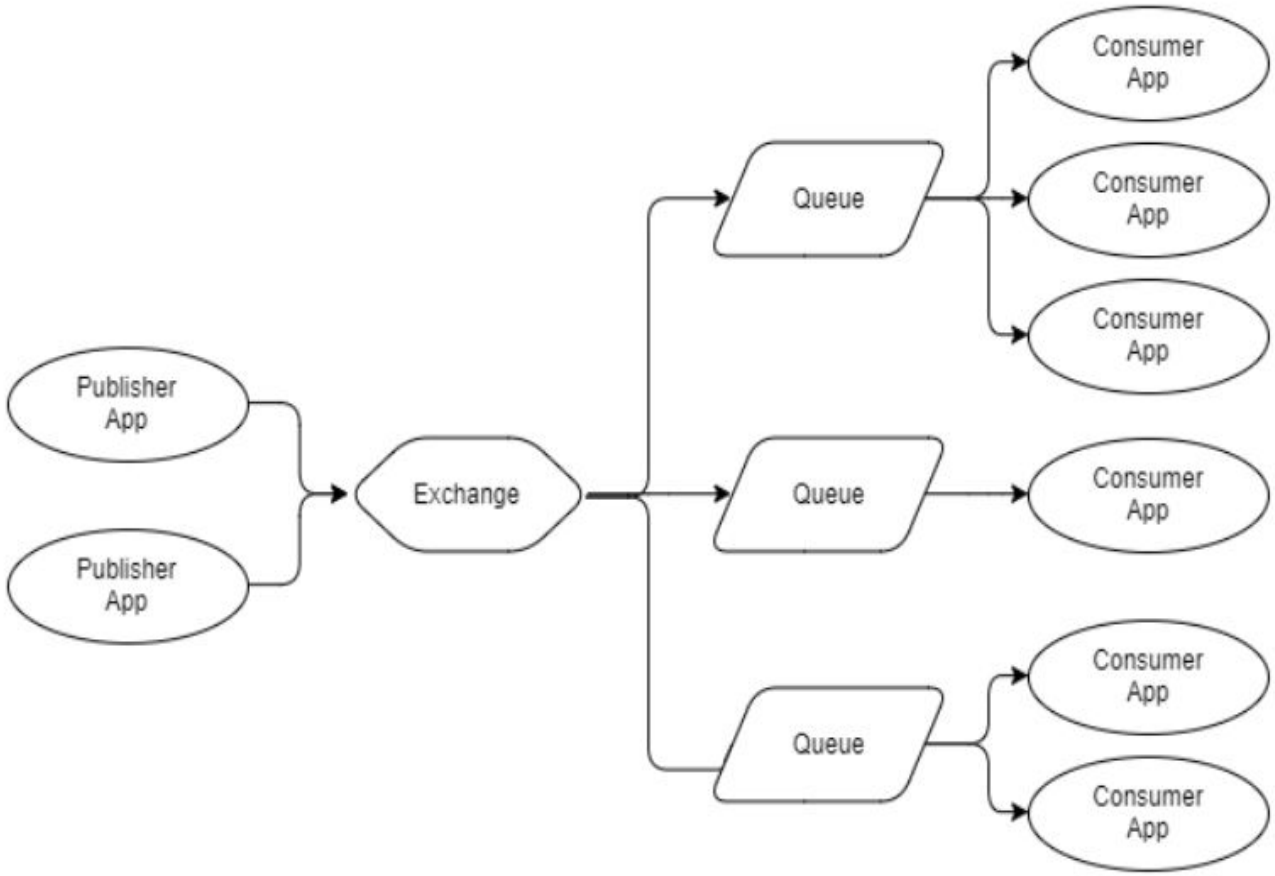
AMQP:

- Apache Qpid
- Apache ActiveMQ
- Azure: Event Hubs, Service Bus
- Solace PubSub+
- JORAM
- StormMQ
- RabbitMQ

Устройство AMQP/RabbitMQ

Термины:

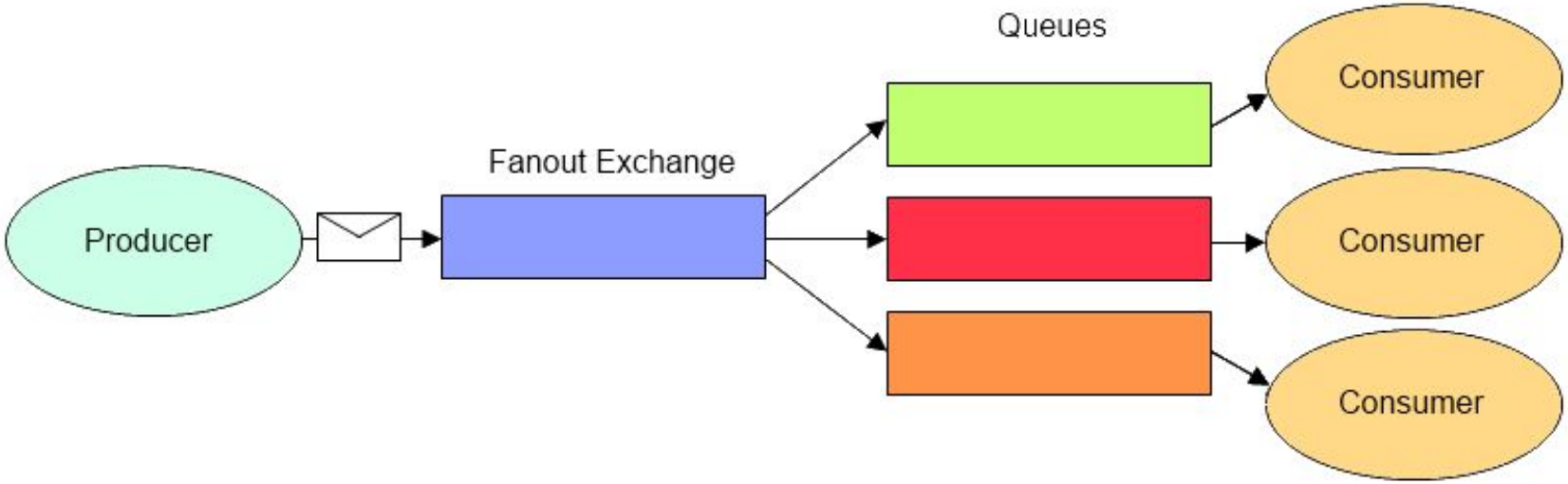
- Producer – отправитель сообщений
- Consumer – получать сообщений
- Channel – виртуальное соединение для экономии физических соединений
- Queue – очередь сообщений
- Exchange – обменник (Fanout, Direct, Topic, Headers)
- Binding – связь обменника и очереди



Устройство AMQP/RabbitMQ

Exchanges:

- **fanout**
- direct
- topic
- headers

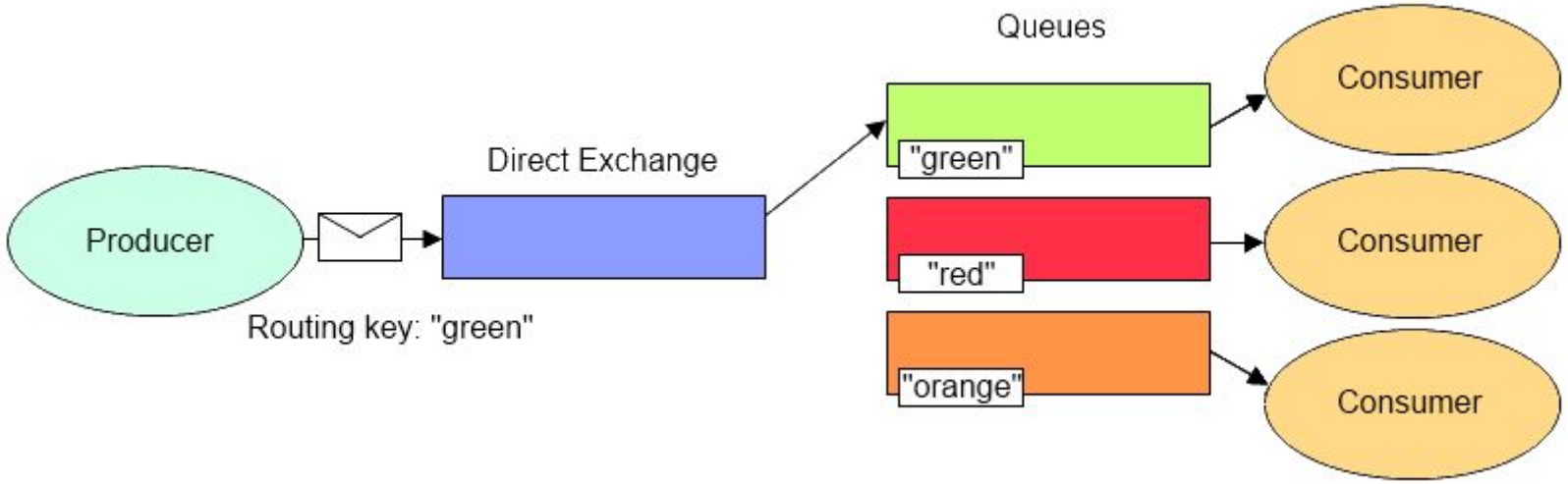


fanout отправляет сообщения во все очереди

Устройство AMQP/RabbitMQ

Exchanges:

- fanout
- **direct**
- topic
- headers

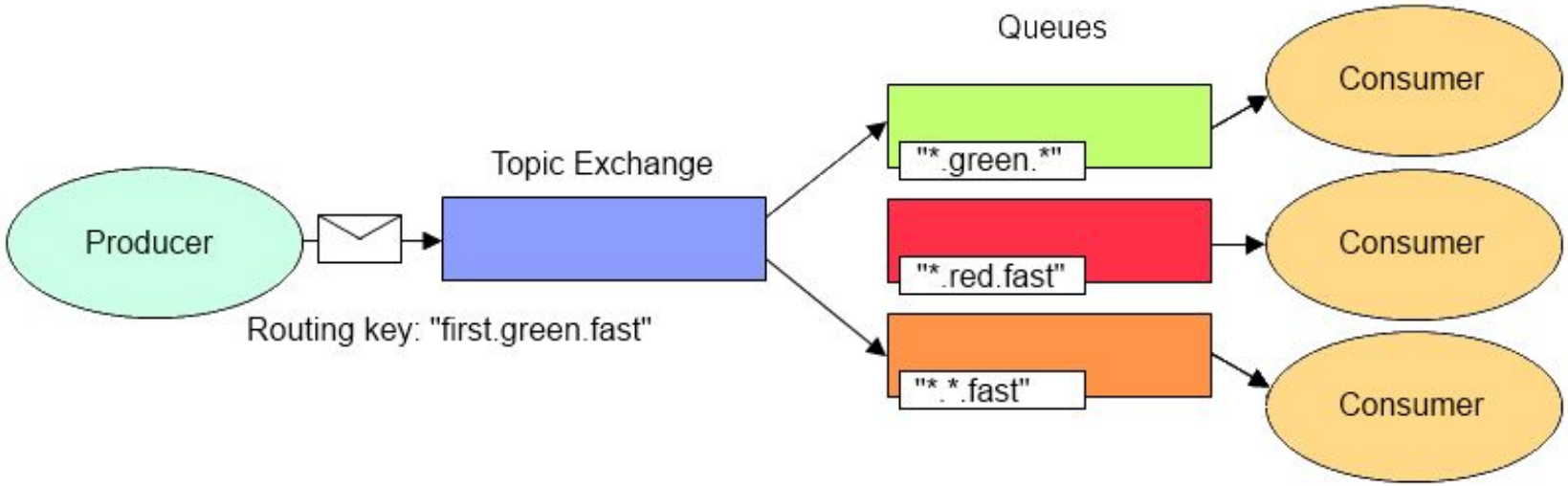


direct отправляет сообщения в конкретную очередь, указанную в routing key

Устройство AMQP/RabbitMQ

Exchanges:

- fanout
- direct
- **topic**
- headers



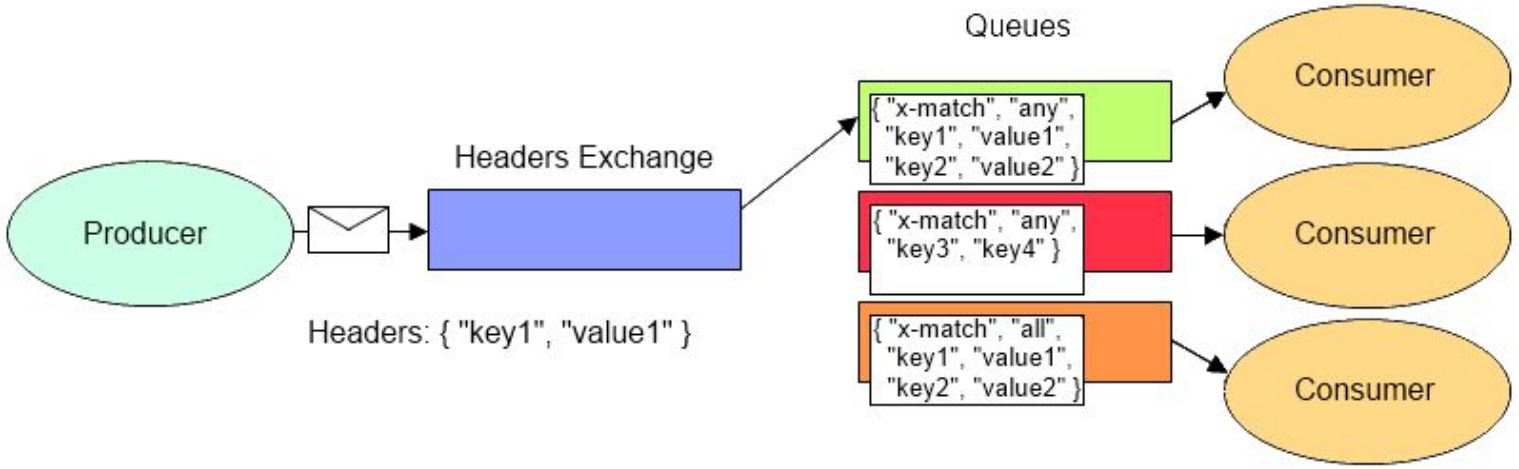
topic отправляет сообщения в очереди, удовлетворяющие маске в routing key

“*.orange.*”, “*.*.rabbit”, “lazy.#”
* (star) - точно одно слово
# (hash) - от нуля и больше слов

Устройство AMQP/RabbitMQ

Exchanges:

- fanout
- direct
- topic
- **headers**



headers отправляет сообщения в очереди, удовлетворяющие binding arguments обменника, согласно заголовкам сообщений

Устройство AMQP/RabbitMQ - API

Создание обменника:

- exchange — название обменника
- type — тип обменника (fanout, direct, etc)
- durable — (true) хранится на диске; (false) временный, удаляется после перезагрузки
- autoDelete — будет удален после diconnect-а всех очередей
- arguments — необязательные аргументы (alternative-exchange, message-ttl, etc)

```
//...
channel.ExchangeDeclare(
    exchange: "my_exchange",
    type: "direct",
    durable: "false",
    autoDelete: "false",
    arguments: null
);
//...
```

Устройство AMQP/RabbitMQ - API

Создание Очереди:

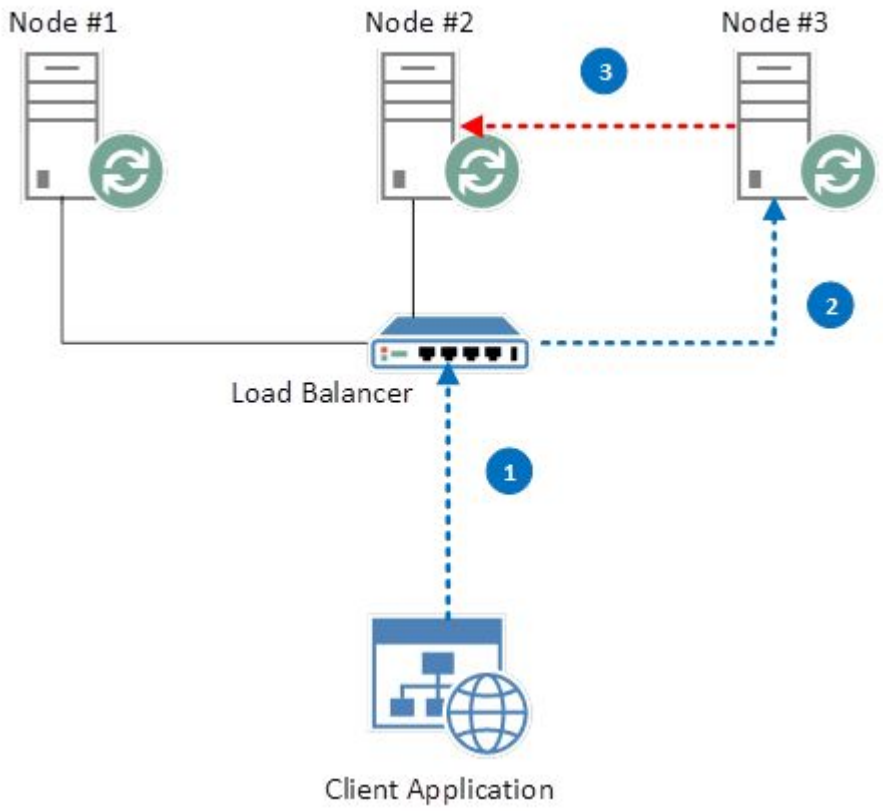
- queue — название очереди
- durable — (true) хранится на диске; (false) временный, удаляется после перезагрузки
- exclusive — не более одного подключения
- autoDelete — будет удален после disconnect-а всех консумеров
- arguments — необязательные аргументы (alternative-exchange, message-ttl, etc)

```
channel.QueueDeclare(queue: "hello",
                     durable: true,
                     exclusive: false,
                     autoDelete: false,
                     arguments: null);
```

Устройство кластера AMQP/RabbitMQ

Кластеризация:

- master-slave архитектура
- Необходимо аккуратно настраивать политику синхронизации
- Неправильно настроенная политика приводит либо к потере сообщений, либо к блокировке всего кластера
- Клиент обязан самостоятельно найти работающий узел RabbitMQ
- Вместо клиента может это сделать балансировщик

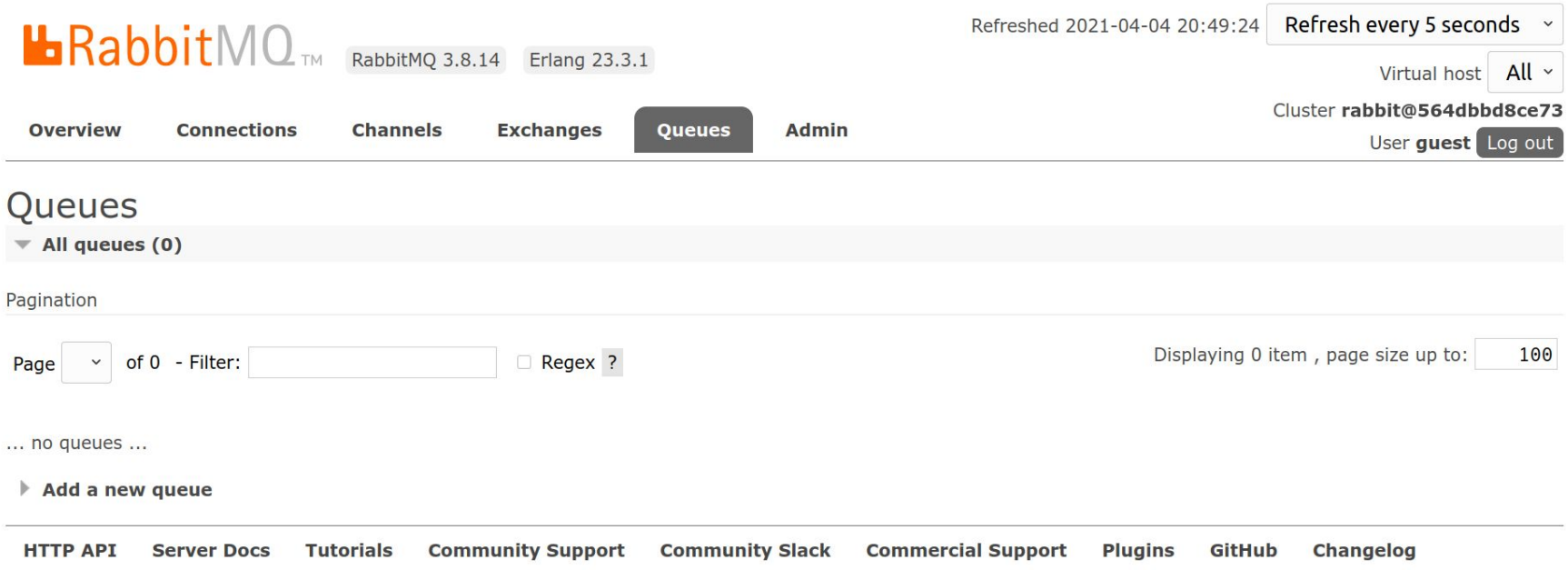


RabbitMQ демо

Запуск RabbitMQ в Docker

DockerHub:

- https://hub.docker.com/_/rabbitmq
- docker run -d -p 5672:5672 -p 15672:15672 --name my-rabbit rabbitmq:3-management
- http://localhost:15672 (user: guest, password: guest)



Работа с RabbitMQ

Библиотеки

Библиотеки:

- https://github.com/rabbitmq/rabbitmq-java-client – официальная библиотека
- spring-boot-starter-amqp – Spring AMQP
- https://github.com/spring-cloud/spring-cloud-stream - Spring Cloud Stream
- ~~https://github.com/JUtupe/ktor-rabbitmq – KTOR~~
- https://github.com/dimensional-fun/usagi Kotlin Multiplatform

Библиотеки: Original

RabbitMQ:

- Создать фабрику
- Добавить соединение
- Создать канал
- Создать очередь
- Подписаться на очередь

```
implementation("com.rabbitmq:amqp-client:5.13.1")
```

```
import com.rabbitmq.client.DeliverCallback
import com.rabbitmq.client.Delivery
import java.nio.charset.StandardCharsets

fun main() {
    val factory = ConnectionFactory()
    val connection = factory.newConnection("amqp://guest:guest@localhost:5672/")
    val channel = connection.createChannel()
    val consumerTag = "SimpleConsumer"

    channel.queueDeclare("test_queue", false, false, false, null)

    println("[$consumerTag] Waiting for messages...")
    val deliverCallback = DeliverCallback { consumerTag: String?, delivery: Delivery ->
        val message = String(delivery.body, StandardCharsets.UTF_8)
        println("[$consumerTag] Received message: '$message'")
    }
    val cancelCallback = CancelCallback { consumerTag: String? ->
        println("[$consumerTag] was canceled")
    }

    channel.basicConsume(QUEUE_NAME, true, consumerTag, deliverCallback, cancelCallback)
}
```

Библиотеки: Spring

Spring:

- Создать приложение
- Создать очереди и обменники
- Создать слушателя

```
@SpringBootApplication
public class HelloWorldMessageApp {
    // ...
}
```

```
@Bean
public Queue myQueue() {
    return new Queue("myQueue", false);
}
```

```
@RabbitListener(queues = "myQueue")
public void listen(String in) {
    System.out.println("Message read from myQueue : " + in);
}
```

Библиотеки: Spring Cloud Stream – более высокий уровень абстракции

Spring:

- Добавляем зависимости (cloud-stream + binder для RabbitMQ)
- Декларируем функции-обработчики
- Увязываем функции с RMQ на уровне конфигурации функций

```
# application.properties
spring.cloud.function.definition=uppercase;javaFunction
```

```
@Configuration
open class DemoKotlinConfiguration {
  @Bean
  open fun uppercase(): (String) -> String {
    return { it.toUpperCase() }
  }

  @Bean
  open fun javaFunction(): Function<String, String> {
    return Function { x -> x }
  }
}
```

Тестирование приложения с RabbitMQ

TestContainers:

- Создать объект тест-контейнера с использованием docker-образа
- Назначить порты и другие параметры подключения
- Стартовать
- использовать в тесте getMappedPort(port)

https://www.testcontainers.org/modules/rabbitmq/

```kotlin
internal class RabbitContainerTest {
    companion object {
        val container by lazy {
            RabbitMQContainer( dockerImageName: "rabbitmq:latest").apply { this: RabbitMQContainer
                withExposedPorts( ...ports: 5672, 15672)
                start()
            }
        }
        val testPort by lazy {
            container.getMappedPort( originalPort: 5672)
        }
    }

    @Test
    fun queueTest() {
        ConnectionFactory().apply { this: ConnectionFactory
            host = "localhost"
            port = testPort
        }.newConnection().use { connection ->
            connection.createChannel().use { channel ->
                var inMessage = ""
                channel.queueDeclare( queue: "testQueue",  durable: false,  exclusive: false,  autoDelete: false,  arguments: null)
                channel.basicConsume(
                    queue: "testQueue",
                    autoAck: true,
                    DeliverCallback{tag, message -> inMessage = String(message.body, Charsets.UTF_8) },
                    CancelCallback {  }
                )
                channel.basicPublish( exchange: "",  routingKey: "testQueue",  props: null, "hello".toByteArray(Charsets.UTF_8))
                Thread.sleep( millis: 1000)
                assertEquals( expected: "hello", inMessage)
            }
        }
    }

}
```

LIVE

Рефлексия

Слайд с заданием

1. Какие основные элементы есть в RabbitMQ?
2. Как происходит доставка сообщений через обменник?
3. Как происходит подключение RabbitMQ в коде?

Тайминг: 15 мин

Тезисы

Подведем итоги

1. RabbitMQ – это брокер сообщений по протоколу AMQP
2. Основные элементы AMQP – Producer, Consumer, Queue, Exchange
3. Типы Exchange: direct, fanout, topic, headers
4. Кластеризация по master-slave, доступ через балансировщик
5. Тестирование через TestContainers

Домашнее задание

1.	Выбрать фреймворк для выпускного проекта и реализовать приложение на нем
2.	Подключить транспортные модели к приложению
3.	Реализовать автоматические тесты

Срок: 09.09.2025

Проверка достижения целей

После занятия вы:

1. Будете знать особенности RabbitMQ и сможете принимать решение о его применении в проекте
2. Узнаете, как взаимодействовать с RabbitMQ из кода
3. Сможете интегрировать транспортную часть, взаимодействующую с RabbitMQ и бизнес-логику в приложении

Рефлексия

С какими впечатлениями уходите с вебинара?

Как будете применять на практике то,
что узнали на вебинаре?

Следующий вебинар

02.09.2025

Модуль 5: Фреймворки и асинхронный транспорт

Транспорт с Kafka

- Ссылка на вебинар будет в ЛК за 15 минут
- Материалы к занятию в ЛК — можно изучать
- Обязательный материал обозначен красной лентой

Список материалов для изучения

1. https://kotlinlang.org/
2. https://gradle.org/
3. https://github.com/otuskotlin/202505-ok-marketplace
4. https://github.com/JUtupe/ktor-rabbitmq
5. https://medium.com/swlh/async-messaging-with-kotlin-and-rabbitmq-d69df1937b25
6. https://github.com/tonsV2/spring-kotlin-rabbitmq
7. https://www.enterpriseintegrationpatterns.com/patterns/messaging/
8. https://github.com/evgnep/otus-arch/tree/master/mq-rabbit - пример работы с RMQ на Spring

Заполните, пожалуйста, опрос о занятии по ссылке в чате

Спасибо за внимание!
Приходите на следующие вебинары